

FSOC Virtual Camera Tracking Simulator

Coarse Pointing, Acquisition and Tracking (PAT) of an optical beacon
with learned beacon identification

Technical Report · software version 0.1.0 · 27 September 2026

Abstract. Free-space optical communication (FSOC) links need their two terminals to point at each other to within a fraction of a milliradian. Before a fine-steering mirror can take over, a *coarse* stage must find the partner terminal's beacon in a wide field of view, confirm that it is the right light, and keep it near boresight on a gimbal with limited slew rate. This report describes a complete software simulation of that coarse stage. A virtual pan-tilt camera renders a synthetic sky with a blinking beacon, decoy lights, stars, clouds, atmospheric turbulence, platform vibration and sensor noise. A closed loop of blob detection, beacon identification, Kalman tracking and PID/feed-forward gimbal control acquires and tracks the beacon, and every frame is logged against ground truth. Identification combines a classical blink-frequency test with *BeaconNet*, a 62,818-parameter convolutional network that classifies a 0.8 s video clip of each candidate; on held-out scenes it reaches ROC-AUC 0.976 versus 0.903 for the blink test alone. In a 40-seed evaluation on random scenes the hybrid identifier acquires the true beacon in 92% of runs (brightest-dot baseline: 62%) and reduces runs with sustained false locks from 42% to 8%, at a processing cost of about 8 ms per frame. The report documents every model and algorithm as implemented, the evaluation method and results, and the known limitations.

Contents

1. Introduction	2
2. System Architecture	2
3. Virtual World and Camera Model	3
4. Disturbance Models	5
5. Candidate Detection	6
6. Beacon Identification	6
7. Training the Network	8
8. Tracking	9
9. Pointing Control and Search	10
10. Performance Monitoring	11
11. Results	11
12. User Interfaces and Tooling	14
13. Limitations and Future Work	14
14. Conclusion	15

1. Introduction

An FSOC terminal transmits data on a narrow laser beam, typically a few hundred microradians wide or less. Link establishment follows the **PAT** sequence: *pointing* the terminal open-loop at the partner's reported position (from GPS or ephemeris data), *acquisition* of the partner's beacon by searching the uncertainty region, and closed-loop *tracking*. Tracking is normally split into a coarse loop (camera plus gimbal, wide field of view, low bandwidth) and a fine loop (quadrant detector plus fast-steering mirror, narrow field, high bandwidth). The coarse loop's job is to bring the beacon inside the fine loop's capture range and keep it there.

The coarse loop faces problems that are hard to reproduce in the field and hard to test repeatably: atmospheric scintillation that makes the beacon twinkle and fade, angle-of-arrival jitter, platform vibration, clouds that hide the beacon for seconds, sun glints and glare, sensor noise, hot pixels, and other lights of similar brightness (aircraft, street lamps, other terminals) that the tracker must *not* lock onto. This project provides a software test bench for all of them.

1.1 Objectives

- Simulate a virtual pan-tilt camera looking at a moving optical beacon, with physically motivated disturbances that can be switched on, off and scaled individually.
- Implement a complete coarse PAT pipeline: search, detection, identification, Kalman tracking, coasting through occlusions, re-acquisition and gimbal control.
- Recognise the designated beacon among look-alike lights from its temporal signature (a 4 Hz blink code), using both a classical signal-processing test and a trained neural network.
- Measure performance against ground truth (acquisition time, lock retention, pointing error, false locks, processing time) and report it automatically for every run, with reproducible seeds.
- Offer a desktop application for interactive demonstration and a headless mode for batch evaluation.

1.2 Scope of this report

Sections 2–9 describe the implementation module by module, with the equations and parameter values that the code actually uses. Section 10 defines the metrics, Section 11 presents measured results, Section 12 the user interfaces and tooling, and Section 13 the limitations. All angles are in degrees unless stated otherwise; with the default camera one pixel corresponds to $0.0125^\circ = 0.218$ mrad.

2. System Architecture

The code base (about 4,600 lines of Python, of which roughly 2,300 form the simulation core in the `fsoc` package) is organised as small single-purpose modules wired together by one GUI-independent class, `Simulation`. One call to `Simulation.step()` produces one camera frame and advances the closed loop by one time step (Figure 1).

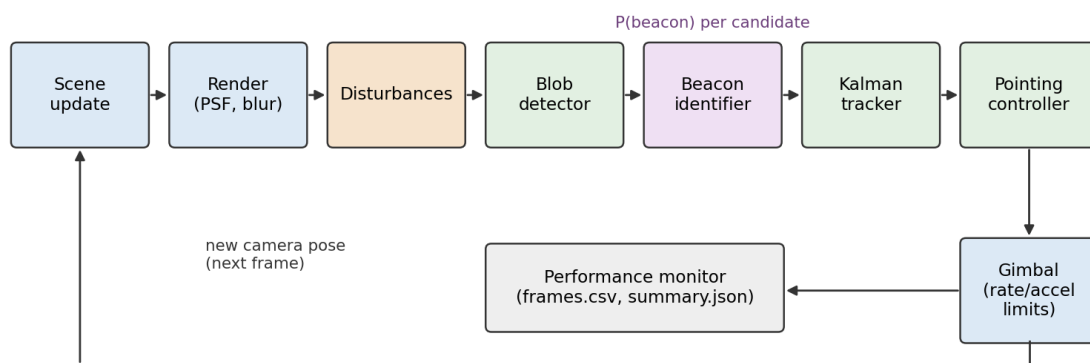


Figure 1. Per-frame closed loop implemented in `fsoc/simulation.py`. The gimbal command computed from frame k moves the camera during the interval before frame $k+1$.

Table 1. Modules of the simulator.

Module	Responsibility
config.py	Loads config/default.yaml and deep-merges a scenario file over it (dicts merge key by key; lists such as the target list are replaced).
targets.py	Beacon and decoys; trajectories: static, linear, Lissajous, circular, random walk; on/off blink modulation.
scene.py	Renders only the visible part of the sky: stars and targets as Gaussian PSFs with per-source motion blur.
camera.py	Pan-tilt camera: field of view, small-angle projection, slew-rate and acceleration limits, travel stops.
disturbances.py	Vibration, turbulence (scintillation, wander, blur, shimmer), clouds, dropouts, glints, glare, sensor noise.
detector.py	Adaptive threshold, region-of-interest threshold, connected components, sub-pixel centroid.
identification.py	Candidate tracklets, blink SNR test, CNN scorer (ONNX via OpenCV DNN), evidence accumulation.
tracker.py	Constant-velocity Kalman filter, gating, SEARCH / ACQUIRING / TRACKING / COASTING state machine, blacklist.
controller.py	PID with velocity feed-forward while tracking; predictive spiral and raster search otherwise.
metrics.py	Ground-truth metrics per frame, run summary, frames.csv / summary.json / summary.txt.
randomizer.py	Random scene generation from the run seed.
visualization.py, gui/	OpenCV overlay; PySide6 desktop application with 3D view and live plots.
tools/	Dataset generation, CNN training and ONNX export, method comparison, multi-seed evaluation.

2.1 Timing and causality

The simulation runs at a fixed rate of 30 frames/s ($\Delta t = 33.3$ ms). Within a step the world advances first, then the frame is rendered from the camera's *actual* line of sight (commanded pose plus vibration offset). Detections are converted to world angles with the *commanded* pose, because a real tracker only knows its encoder readings; vibration therefore appears to the tracker as apparent target motion, as it would in hardware. The controller output is applied to the gimbal after the frame, which introduces the one-frame latency of a real camera-in-the-loop system.

2.2 Configuration and reproducibility

Every parameter lives in config/default.yaml, grouped by module, and every disturbance has an enabled switch and a strength multiplier. Six scenario files list only the values that differ from the defaults. Each run uses a single integer seed: with seed: null a fresh seed is drawn and written to the report, so any run can be replayed exactly with --seed. The scene randomiser uses its own generator derived from the same seed, so the scene and the noise are both reproducible.

3. Virtual World and Camera Model

3.1 Coordinates and projection

The world is a patch of sky of $\pm 30^\circ$ in azimuth and $\pm 20^\circ$ in elevation. The camera has a 640×480 sensor and an $8^\circ \times 6^\circ$ field of view, giving 80 pixels per degree on both axes. Because the field of view is small, the projection is linear (small-angle): for a camera pointing at (az_c, el_c)

$$x = c_x + (az - az_c) \cdot ppd_x \quad y = c_y - (el - el_c) \cdot ppd_y$$

with (c_x, c_y) the image centre and ppd the pixels per degree. The inverse mapping converts detections back to world angles. Working in world coordinates is central to the design: target motion and camera motion are separated, so the tracker's velocity estimate is the target's angular velocity and not an artefact of panning.

3.2 Gimbal dynamics

The camera is driven by angular-rate commands. Each axis first limits the change in rate to a $a_{\max} \cdot \Delta t$ ($80^\circ/\text{s}^2$) and then clips the rate to $\pm 25^\circ/\text{s}$, then integrates the position. Hard stops at the world boundaries zero the rate. The controller therefore cannot teleport the camera; the search pattern and the tracking gains have to respect these limits.

3.3 Rendering

Instead of cropping a large pre-rendered image, `Scene.render` draws only the sources that fall inside the current view. Each source is a 2-D Gaussian point-spread function at a sub-pixel position ($\sigma = 2$ px for targets, 0.7 px for stars), evaluated only in a $\pm 3\sigma$ window. Six hundred background stars with a power-law brightness distribution (peak 10–60 grey levels, below the detection floor) provide realistic clutter that moves across the image when the camera pans.

Motion blur is computed per source from its motion *relative to the camera* during the exposure (10 ms). The source is drawn as a Gaussian ‘line’ along the streak vector $u = (v_{\text{src}} - v_{\text{cam}}) \cdot \text{ppd} \cdot t_{\text{exp}}$. To conserve flux the peak is reduced to

$$\text{peak}_{\text{eff}} = \text{peak} / (1 + L / (\sqrt{2\pi} \sigma))$$

where L is the streak length in pixels. A target the camera follows therefore stays sharp while stars streak, exactly as in a real tracking camera, and the beacon smears and dims during fast search slews. Turbulence blur widens σ to $\sqrt{\sigma^2 + \sigma_b^2}$ and scales the peak by $(\sigma/\sigma')^2$, again conserving flux.

3.4 Targets and trajectories

Every light source is a `Target` with an intensity, a spot size, an optional blink (frequency, duty cycle, off level, phase) and a trajectory. Exactly one target must be the beacon. The default beacon blinks at **4 Hz with a 70% duty cycle and dims to 30% when ‘off’**, so it never disappears completely yet carries a clear temporal code. Five trajectory types are available (Table 2).

Table 2. Trajectory models.

Type	Model
static	Fixed position.
linear	Constant velocity, reflected at the world edges.
lissajous	$az, el = c + A \cdot \sin(2\pi ft + \phi)$ per axis; smooth figure-of-eight motion.
circular	Orbit of radius r and period T about a centre (e.g. a loitering UAV).
random_walk	Constant speed; the <i>turn rate</i> is an Ornstein–Uhlenbeck process (σ, τ), so the target banks into and out of turns. Near the edge it commits to a turn away from the nearest wall at up to 3 rad/s; the edge margin is at least 2.2 turning radii so that a U-turn always fits.

3.5 Random scenes

With `randomize.enabled` (the default for interactive use) each run draws a beacon path type and speed (0.8–5 $^\circ/\text{s}$, with the path parameters solved so that the peak speed matches), a beacon intensity, 0–5 decoys (half of them blinking at deliberately wrong rates: 0.5–2.5 Hz or 6–10 Hz), turbulence strength 0.5–2.0, vibration strength 0.3–2.0, clouds with probability 0.5, and an **initial pointing error** of 3–10 $^\circ$ in a random direction. The last item models the open-loop pointing step of PAT: the camera starts aimed at the partner's reported position, which is wrong by a few degrees, and must search from there.

4. Disturbance Models

All stochastic disturbances use first-order Gauss–Markov (discrete Ornstein–Uhlenbeck) processes rather than independent per-frame noise, because real turbulence and jitter are temporally correlated:

$$x_{k+1} = a \cdot x_k + \sqrt{(1 - a^2) \cdot \sigma} \cdot n_k \quad a = \exp(-\Delta t / \tau), \quad n_k \sim N(0, 1)$$

This keeps the stationary standard deviation equal to σ for any time step, with correlation time τ .

Table 3. Disturbance models. Each has an enabled switch and a strength multiplier.

Effect	Physical cause	Implementation (default parameters)
Platform vibration	Engine, rotor, structural modes	Two sinusoidal tones (7 Hz, 0.010°; 23 Hz, 0.005°) plus broadband OU jitter (0.006°, $\tau = 30$ ms) added to the line of sight. Its rate also feeds motion blur.
Scintillation	Refractive-index turbulence	Log-normal intensity gain $\exp(\chi - \sigma^2/2)$ with unit mean; χ is OU with $\sigma = 0.3$, $\tau = 30$ ms, independent per source.
Beam wander	Angle-of-arrival jitter	Per-source OU position offset, 0.8 px, $\tau = 0.1$ s.
Turbulence blur	Beam spreading	Extra PSF width 0.6 px added in quadrature.
Heat shimmer	Local image distortion	OU displacement field on a 16×12 grid (0.6 px, $\tau = 0.15$ s), cubic upsampling, applied with <code>cv2.remap</code> .
Clouds	Drifting cloud banks	Gaussian optical-depth blobs moving with the wind and wrapping around; transmission $T = \exp(-\tau)$; image = (img - b) · T + b with cloud glow b = 35.
Dropouts	Obstruction, beacon power dips	Poisson events (mean interval 15 s) that blank the beacon for 0.5–2 s.
Glints	Sun reflections, flares	Poisson flashes (0.3 Hz) inside the FOV lasting 0.05–0.4 s, peak 150–255.
Glare	Sun or moon near the FOV	Wide Gaussian background (level 120, $\sigma = 3^\circ$) fixed in the world.
Sensor	Camera electronics	Shot noise $\sigma = 0.5 \sqrt{\text{signal}}$ plus read noise 3 grey levels, 25 hot pixels, Poisson single-pixel hits; 8-bit saturation.

Order matters and follows the physics: per-source effects (scintillation, wander, blur, dropouts) are applied while rendering each source, then whole-image effects are applied to the rendered frame: clouds, glare, shimmer and finally sensor noise. For speed, the sensor model draws one bank of unit Gaussian noise twice the frame size at start-up and takes a random crop each frame; visually this is indistinguishable from fresh noise and removes the most expensive per-frame operation. Turbulence, vibration and sensor strength can be changed while a run is in progress (the GUI sliders); a disturbance that was off is created on demand.

A frame counts as **occluded** when a dropout is active or cloud transmission at the beacon is below 50%, at which point the dimmed beacon falls below the detection threshold. Occluded frames are reported separately so that the tracker is not blamed for what no camera can see.

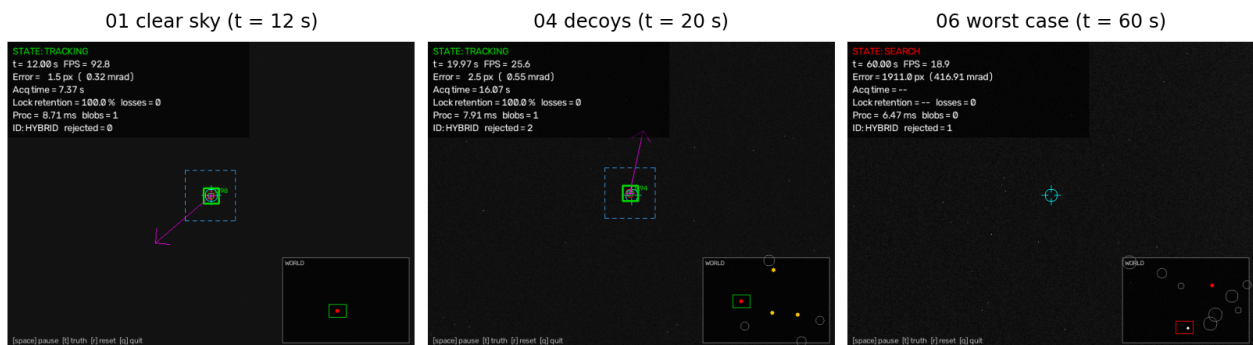


Figure 2. Frames produced by the simulator with the OpenCV overlay (truth markers on). Left: clear sky, locked, 1.5 px error. Centre: decoy scenario, locked on the beacon (green box, score 0.94) with three decoys visible on the world minimap (orange). Right: worst-case scenario with seed 42, still searching at 60 s; clouds (grey circles on the minimap), noise and glare are visible.

5. Candidate Detection

The detector (BlobDetector) finds every bright spot in the 8-bit frame and leaves the decision of *which* spot is the beacon to later stages.

- **Smoothing:** a 3×3 Gaussian blur suppresses single-pixel noise and hot-pixel hits.
- **Adaptive threshold:** background b = median and robust noise $\sigma_n = 1.4826 \cdot \text{MAD}$ are estimated on a $4 \times$ sub-sampled image; threshold = $\max(b + 8\sigma_n, 80)$. The median and MAD are barely affected by the few bright spots, and sub-sampling makes the estimate cheap.
- **Region of interest:** inside an 80×80 px window around the tracker's predicted position a much lower threshold is used, $\max(b + 5\sigma_n, 45)$. The beacon's blink-off phase (30% power) and deep scintillation fades are still detected there, while the rest of the image is not flooded with stars and noise.
- **Connected components** (8-connectivity) with an area filter of 2–400 px.
- **Sub-pixel centroid** weighted by brightness above background on the *un-blurred* frame, plus peak and integrated flux per blob.

Detections are then converted to world angles with the commanded camera pose.

6. Beacon Identification

In a single frame the beacon, the decoys and bright stars are indistinguishable Gaussian spots. What differs is their behaviour over time: the beacon blinks at a known rate (its 'code'), decoys are steady or blink at other rates, glints vanish within a fraction of a second, and hot pixels stay fixed on the sensor while the sky moves. Identification therefore works on short histories of each candidate.

6.1 Candidate tracklets

CandidateTracker follows *every* detection from frame to frame in world coordinates with greedy nearest-neighbour association (closest pairs first). Each tracklet predicts with a smoothed velocity; its association gate starts at 0.25° and widens by 50% per missed frame (capped at 1°) because a blinking or fading source becomes less certain while unseen. Every frame, *including frames in which the candidate was not detected*, a 24×24 px patch is cut at the predicted position with sub-pixel interpolation (`cv2.getRectSubPix`), and the aperture flux (7×7 box minus the patch median) is stored. Recording the off-frames is what makes the blink visible. A tracklet is 'ready' once it holds 24 patches (0.8 s); the flux history is kept twice as long (1.6 s) for sharper frequency analysis.

6.2 Classical blink test

BlinkScorer evaluates a Hann-windowed single-bin DFT of the flux history at the beacon frequency $f_0 = 4$ Hz and on a 0.25 Hz grid of 'noise' frequencies that excludes ± 1 Hz around f_0 and $2f_0$. The test statistic is a spectral signal-to-noise ratio

$$\text{SNR} = P(f_0) / \text{mean } P(f_{\text{noise}}) \cdot \min(1, P(f_0) / \max_{k \in \{2,3,4\}} P(f_0/k))$$

Broadband flicker such as scintillation raises signal and noise bins equally and cancels out. The second factor handles **harmonic decoys**: a light blinking as a square wave at $f_0/2$ has a harmonic exactly at f_0 ; such a light has more power at its fundamental than at f_0 and is penalised. The SNR is mapped to a probability with a logistic calibrated on validation data, where the beacon's median SNR was about 7 and every decoy type's median 0.5–1.5:

$$p = 1 / (1 + \exp(-4.2 \cdot (\log_{10} \text{SNR} - \log_{10} 3)))$$

so SNR 1 $\rightarrow$ 0.1, SNR 3 $\rightarrow$ 0.5 and SNR 10 $\rightarrow$ 0.9. The DFT matrices are cached per window length, so scoring all candidates is one small matrix product.

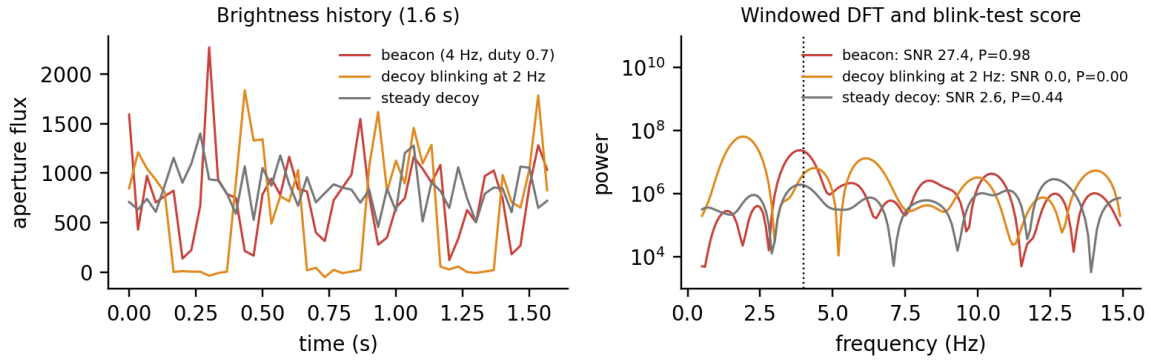


Figure 3. Blink test on synthetic 1.6 s flux histories with scintillation ($\sigma = 0.3$) and additive noise, scored with the project's BlinkScorer. The beacon's 4 Hz peak gives a high SNR; the 2 Hz decoy is rejected by the harmonic check; a single clip of a steady light can reach an intermediate score, which is why single-clip scores are accumulated over time (Section 6.5).

6.3 BeaconNet: learned identification

CNNScorer runs a small convolutional network that treats the candidate's last $T = 24$ patches as a 24-channel image, i.e. a short video clip. The first convolution therefore learns spatio-temporal filters that combine the blink pattern, the spot shape and the consistency of the motion in one step. Before inference each clip is normalised by subtracting its median and dividing by its maximum (at least 20), then clipped to $[-0.5, 1.5]$; this keeps the *relative* temporal pattern while making the network insensitive to absolute beacon power or cloud dimming.

Table 4. BeaconNet architecture (tools/train_classifier.py).

Layer	Output shape	Parameters
Input clip ($T = 24$ frames)	$24 \times 24 \times 24$	–
Conv 3×3 (24→32) + BatchNorm + ReLU, MaxPool 2	$32 \times 12 \times 12$	7,008
Conv 3×3 (32→64) + BatchNorm + ReLU, MaxPool 2	$64 \times 6 \times 6$	18,624
Conv 3×3 (64→64) + BatchNorm + ReLU	$64 \times 6 \times 6$	37,056
Global average pool, Dropout 0.3, Linear (64→2)	2 logits	130
Total		62,818

At run time the model is loaded from `models/beacon_cnn.onnx` with OpenCV's DNN module, so PyTorch is not needed to run the application. All ready tracklets are scored in one batch; a softmax over the two logits gives $P(\text{beacon})$. If the model file is missing, the simulation prints a warning and falls back to the blink test.

6.4 Methods

Four identification methods are selectable: **hybrid** (default: the mean of CNN and blink probabilities), **cnn**, **blink**, and **none** (no identification: the tracker takes the brightest candidate, the baseline). The hybrid is the default because on validation scenes it made the fewest false 'this is the beacon' calls: the CNN is better at recognising the beacon, the blink test better at rejecting wrong-rate decoys (Table 7), and averaging lets each veto the other's mistakes.

6.5 Evidence accumulation

A single 0.8 s clip can be misleading (a deep fade, a blink hidden in noise). Each clip probability, clamped to $[0.02, 0.98]$, is converted to log-odds ℓ and averaged into a per-tracklet evidence value with a leaky running mean

$$e_n = e_{n-1} + \alpha(\ell_n - e_{n-1}), \quad \alpha = \max(1/n, 1/90), \quad \text{score} = 1/(1 + e^{-e_n})$$

This is an exact mean for the first 90 clips and an exponential memory of about 3 s afterwards: consistent evidence builds up quickly, single outliers barely move the verdict, and a verdict can still change if the tracker has been following the wrong light. A detection reports its score only after at least 24 scored clips (1.6 s of flux history, enough to tell 2 Hz from 4 Hz); before that its score is 0.5 ('undecided').

7. Training the Network

7.1 Dataset generation

Labels come for free because the simulator knows where every source is. `tools/generate_dataset.py` runs randomised episodes (15 s each) using the *same* rendering, detection and candidate-tracking code as the application. Each episode randomises the beacon's waveform (duty 0.6–0.8, off level 15–45%), intensity, spot size and trajectory; adds 2–6 decoys drawn from four types (steady; clearly different rate 0.4–2.5 or 5.5–12 Hz; near rate 2.5–3.0 or 5.0–5.5 Hz; harmonic at $f_0/2$, $/3$ or $/4$); and randomises every disturbance (turbulence up to 3×, vibration up to 2.5×, glints up to 3 Hz, clouds, glare and dropouts with some probability). A virtual operator moves the camera around a focus point that switches between the beacon and random decoys, with a random wander range and speed, so both steady tracking and fast slewing are represented.

Every fourth frame, each ready tracklet becomes a sample: its 24-patch clip, the label (beacon or not) and the kind of source it sat on. A tracklet is matched to the nearest true source within 0.12° ; samples whose window is less than 80% consistent are discarded as ambiguous. Train and validation sets are split **by episode**, so validation scenes are never seen in training. Episodes run in parallel worker processes.

Table 5. Generated datasets (sample counts by source kind). ‘Other’ = stars, hot pixels and noise.

Dataset	Samples	Validation	Beacon	Steady	Wrong rate	Near rate	Glint	Harmonic	Other
v1 (beacon_dataset.npz)	17,953	4,281	4,506	2,636	2,044	735	0	0	8,032
v2 (beacon_dataset_v2.npz)	16,218	2,467	4,764	1,760	1,912	733	4	759	6,286

7.2 Training procedure

Training uses PyTorch with class-weighted cross-entropy (to balance the minority beacon class), AdamW (learning rate $2 \cdot 10^{-3}$, weight decay 10^{-4}) with a one-cycle schedule, batch size 128 and 25 epochs. Augmentation applies random horizontal and vertical flips, 90° rotations (the problem is rotation invariant) and small additive noise. The checkpoint with the lowest validation loss is kept, exported to ONNX (opset 13, dynamic batch) and checked against OpenCV's DNN: the maximum logit difference is $9.5e-07$, i.e. the deployed model reproduces PyTorch exactly. By default a retrain writes to `models/candidate/` so that it never silently replaces the model the application uses.

7.3 Validation results

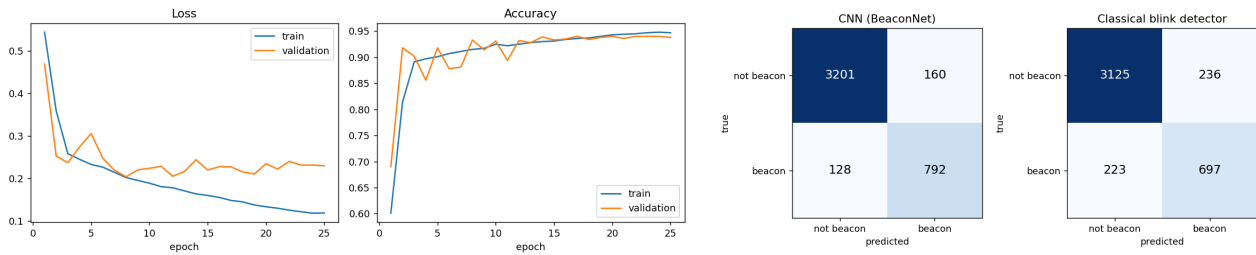
Table 6. Per-clip classification on 4,281 held-out validation clips (shipped model, `models/training_report.json`). Single clips, before evidence accumulation.

Metric	BeaconNet (CNN)	Classical blink test
Accuracy	0.933	0.893
Precision	0.832	0.747
Recall	0.861	0.758
F1 score	0.846	0.752
ROC-AUC	0.976	0.903
Confusion (TP / FP / TN / FN)	792 / 160 / 3201 / 128	697 / 236 / 3125 / 223

Table 7. Accuracy per candidate type on the validation set. The two methods fail on different kinds of candidate, which motivates the hybrid.

Candidate type	n	CNN accuracy	Blink accuracy
beacon	920	0.861	0.758
steady decoy	404	0.859	0.856
wrong rate decoy	575	0.850	0.969
near rate decoy	135	0.963	0.933
other	2247	0.995	0.933

The CNN is clearly better at recognising the beacon (recall 0.86 vs 0.76) and at rejecting stars, hot pixels and noise (0.995 vs 0.933); the blink test is better on decoys blinking at a clearly wrong rate (0.969 vs 0.850). The CNN costs about 0.29 ms per candidate on a CPU. These are single-clip figures; in the closed loop the accumulated score over up to 3 s is far more reliable.

**Figure 4.** Left: training and validation loss and accuracy per epoch. Right: validation confusion matrices of BeaconNet and of the classical blink detector.

8. Tracking

8.1 Kalman filter

The beacon is tracked in world angles with a 2-D constant-velocity Kalman filter, state $x = [az, el, v_{az}, v_{el}]^T$. The prediction uses F with $F_{02} = F_{13} = \Delta t$ and the discrete white-noise acceleration model per axis

$$Q_{axis} = q^2 \cdot g g^T, \quad g = [\Delta t^2/2, \Delta t]^T$$

with $q = 3 \text{ }^\circ/\text{s}^2$ by default (10–15 $^\circ/\text{s}^2$ in the fast-target scenarios). The measurement is the detection's position with $R = (0.02^\circ)^2 \cdot I$. The initial velocity variance is large (25 $(^\circ/\text{s})^2$) so the filter learns the target's velocity within a few frames.

8.2 Data association and gating

A detection is accepted when it is statistically consistent with the prediction, Mahalanobis distance $d^2 = y^T S^{-1} y < 11.6$ (99.7% for two degrees of freedom), or when it lies within a minimum gate of 0.15° of the prediction. The minimum gate keeps lock through sudden manoeuvres that the constant-velocity model did not expect; it grows by 50% per consecutive miss up to 4×. With identification enabled, a detection that belongs to the tracklet already being followed wins over any other candidate inside the gate. Detections that are blacklisted or 'clearly not the beacon' are removed before association.

8.3 Acquisition state machine

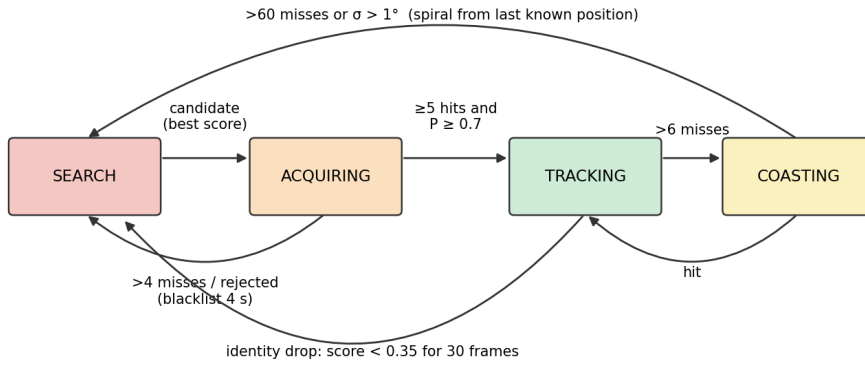


Figure 5. Tracker state machine (fsoc/tracker.py) with the default thresholds.

- **SEARCH:** no track. When candidates exist, the most beacon-like one (highest score, flux breaks ties) starts a tentative track.
- **ACQUIRING (verify):** the candidate is followed and verified. It is confirmed after at least 5 hits *and*, with identification, once its accumulated score is ready and ≥ 0.7 . It is abandoned after more than 4 consecutive misses, and *rejected and blacklisted for 4 s* if its score falls below 0.2 with at least 24 clips of evidence, or if it is not confirmed within 90 frames.
- **TRACKING:** confirmed lock. Up to 6 missed frames (blink-off, scintillation fades) are held without changing state. A lock whose score stays below 0.35 for 30 frames is dropped as a false lock and blacklisted.
- **COASTING:** the beacon is not seen (e.g. behind a cloud); the filter predicts only and the gimbal follows the prediction. A hit returns to TRACKING. Coasting ends after 60 frames or as soon as the position uncertainty exceeds 1° , whichever comes first; fast targets reach the uncertainty limit much sooner. The last position and velocity are handed to the search.

Rejecting a candidate requires more evidence than accepting it, because wrongly blacklisting the true beacon costs 4 s of blindness, while taking slightly longer to confirm costs little. The counters rejections and dropped_locks are included in every report.

9. Pointing Control and Search

9.1 Tracking control law

While a target estimate exists the controller commands the gimbal rate

$$\omega = k_{ff} \cdot \hat{v} + K_p e + K_i \int e dt + K_d \dot{e}$$

where e is the angle between the Kalman estimate and the boresight, $\hat{v}$ the estimated target velocity, $k_{ff} = 1$, $K_p = 5 \text{ s}^{-1}$, $K_i = 0.5 \text{ s}^{-2}$, $K_d = 0.05$ and the integral is clamped to $\pm 2^\circ \cdot \text{s}$ (anti-windup). Velocity feed-forward carries most of the load: the camera moves with the target, and the feedback terms only remove the residual offset. No extra one-frame lead is added to e because the feed-forward term already accounts for motion over the next frame. The integral and derivative states are reset when the search starts.

9.2 Search strategy

In SEARCH the controller flies an Archimedean spiral $r = s \cdot \theta / 2\pi$ centred on the last known beacon position (or the initial pointing direction). The arm spacing s is $0.8 \times$ the smaller FOV dimension (4.8°), so consecutive arms overlap and no sky is skipped; θ is advanced so that the scan point moves at about $12^\circ/\text{s}$, and the camera follows it with a proportional gain of 6 s^{-1} . Three refinements make the search practical:

- **Predictive centre:** the spiral centre drifts along the beacon's last known velocity with a fading horizon, $c(t) = c_0 + v \cdot \tau (1 - e^{-t/\tau})$, $\tau = 3 \text{ s}$, because a lost target keeps moving.
- **Bounded local search:** the spiral restarts beyond a 15° radius; after 4 local passes one full-sky raster ('lawnmower') pass is flown so that every direction is eventually covered, then the local spiral resumes.

- **No reset on false alarms:** during ACQUIRING the spiral state is kept, so a glint that is briefly followed and then abandoned lets the scan resume where it left off instead of restarting.

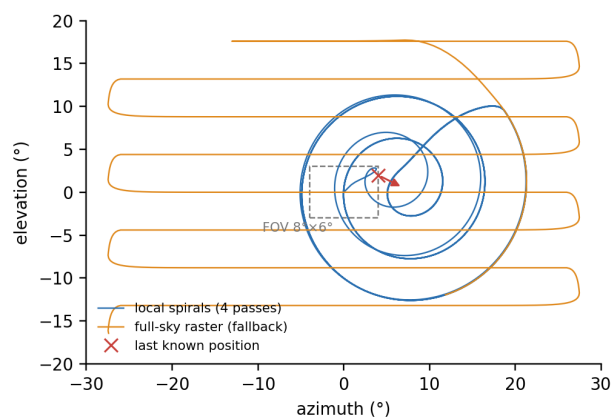


Figure 6. Search path flown by the real controller and gimbal model for 90 s after a loss at $(4^\circ, 2^\circ)$ with last velocity $(1, -0.5)^\circ/\text{s}$: four drifting local spirals (blue), then the full-sky raster fallback (orange). The dashed box shows one field of view.

10. Performance Monitoring

PerformanceMonitor compares the tracker with ground truth every frame and writes a per-frame log and a summary for every run. The definitions are deliberately strict: following a decoy is never counted as lock.

Table 8. Performance metrics (fsoc/metrics.py).

Metric	Definition
Pointing error	Angle between the true beacon and the camera's actual line of sight (including vibration), in degrees, mrad and pixels.
Locked	Tracker in TRACKING and beacon in the FOV and estimate within 8 px of the true beacon.
False lock	Tracker in TRACKING but the estimate is not on the beacon.
Acquisition time	Simulated time until the first locked frame.
Time locked	Locked frames / all frames; also penalises slow or failed acquisition.
Lock retention	Locked frames / frames after first lock; also reported excluding occluded frames.
Handover ready	Share of locked frames with pointing error ≤ 10 px (≈ 2.2 mrad): close enough for a fine-pointing stage.
Processing time	Wall-clock time of detection + identification + tracking + control per frame (rendering timed separately).

Each run folder contains `frames.csv` (21 columns per frame: state, lock flags, true, commanded, actual and estimated positions, errors, detection count, timings), `summary.json` and a human-readable `summary.txt`. The summary also records the seed, the random-scene description, the identification method and the numbers of rejected candidates and identity-dropped locks.

11. Results

11.1 A closed-loop run

Figure 7 shows the decoy scenario (one beacon, two steady decoys and one decoy blinking at 1 Hz, all of similar brightness) with seed 1, run once with the hybrid identifier and once with the brightest-dot baseline. With identification, the tracker rejects 1 candidate during verification, confirms the beacon at 8.7 s and holds it for the rest of the run with no lock losses: 85.5% of the run locked, mean error 2.605 px (0.57 mrad), maximum 8.878 px, and 100% of locked frames inside the handover zone. Without identification the tracker locks onto a steady decoy after the first search and follows it for the entire run (1642 false-lock frames, 0% locked on the beacon), which is exactly the failure the identification stage exists to prevent.

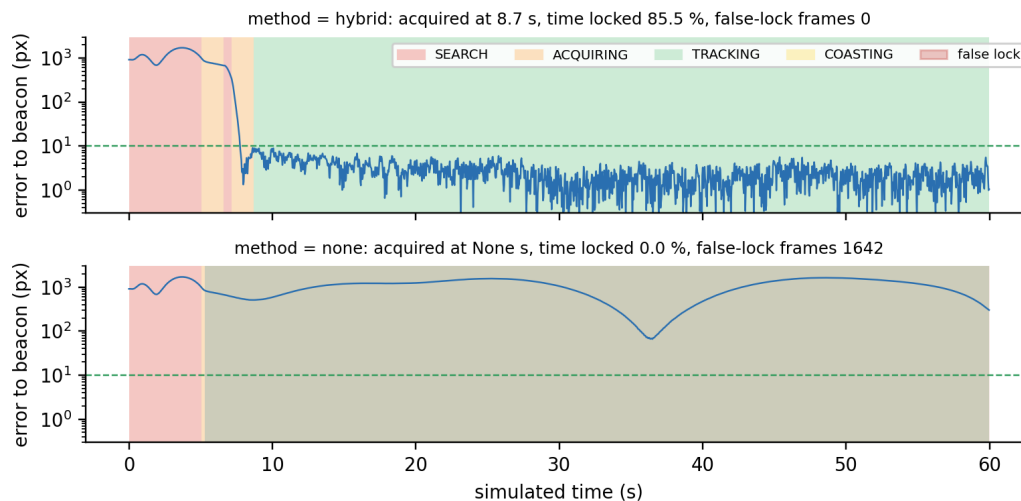


Figure 7. Error between the true beacon and the line of sight (log scale) with the tracker state as background, decoy scenario, seed 1, 60 s. Top: hybrid identification. Bottom: no identification. The dashed line marks the 10 px handover radius.

11.2 Multi-seed evaluation on random scenes

Because a single run can be lucky or unlucky, `tools/evaluate.py` repeats each configuration over many seeds; for random scenes every seed is a different beacon path, decoy set, atmosphere, cloud cover and initial pointing error. Each run lasts 60 s. Table 9 summarises 40 random scenes with and without identification.

Table 9. Random-scene evaluation, 40 seeds per method, 60 s per run (runs/evaluation/results.csv).

Metric	Hybrid identification	None (brightest dot)
Runs	40	40
Beacon acquired [95% Wilson CI]	92% [80–97]	62% [47–76]
Acquisition time, median [IQR]	5.2 s [2.2–16.8]	2.3 s [0.5–4.3]
Time locked, mean $\pm$ 95% CI	71.5 $\pm$ 9.5%	53.7 $\pm$ 14.4%
Runs with $\geq$ 80% of the run locked	55%	50%
Lock retention after acquisition, median	100.0%	100.0%
Runs with a false lock $>$ 1 s	7.5%	42.5%
Mean tracking error while locked, median	3.56 px	4.28 px
Processing time per frame, mean	8.1 ms	3.7 ms

Identification is the single most important element for robustness. It raises the acquisition rate from 62% to 92% and cuts runs with sustained false locks from 42% to 7.5%. The price is a slower first lock (median 5.2 s vs 2.3 s), because a candidate must accumulate at least 1.6 s of evidence and reach a score of 0.7 before it is confirmed, and about twice the processing time, which at 8 ms still leaves ample margin inside the 33 ms frame budget. Once acquired, the lock is typically held for the rest of the run (median retention 100%).

Breaking the hybrid results down by scene property (runs/evaluation/summary.md) shows where the remaining difficulty lies: time locked falls from about 81% for beacons slower than 3.5 $^{\circ}$ /s to 60% for faster ones, and from 73–78% at turbulence $\leq$ 1.5 to 63% above. The number of decoys matters little with identification (67–83% across bins), whereas without it time locked collapses from 71% with no decoys to 39% with three or more.

11.3 Fixed scenarios

Table 10. Fixed test scenarios with the hybrid identifier, 60 s runs over seeds 1–10 (60 runs in total). The scenario defines the scene; the seed varies noise, turbulence and clouds.

Scenario	Runs	Acquired	Acq. time median	Time locked mean	Retention median (visible)	Beacon occluded	Error median
01 clear sky	10	100%	7.4 s	87.7%	100.0% (100.0%)	0.0%	1.44 px
02 strong turbulence	10	100%	11.4 s	52.4%	97.1% (97.3%)	9.1%	4.59 px
03 occlusion	10	100%	27.5 s	31.6%	54.5% (84.0%)	48.0%	3.86 px
04 decoys	10	100%	15.1 s	63.7%	100.0% (100.0%)	7.7%	2.70 px
05 fast target	10	100%	20.3 s	63.1%	99.3% (99.3%)	0.0%	9.63 px
06 worst case	10	90%	26.8 s	9.7%	29.5% (32.5%)	25.8%	8.23 px

The **clear-sky** baseline locks in every run with about 1.4 px mean error; its time locked ($\sim 88\%$) is limited only by the search and verification time at the start. **Strong turbulence** ($2.5\times$ turbulence and vibration, $1.5\times$ sensor noise) still acquires every time, but deep fades and large apparent jitter cause lock losses and brief false locks, so time locked varies widely between seeds. **Occlusion** is dominated by the environment: the beacon is hidden for almost half of every run, yet retention over the *visible* frames remains high, showing that coasting and predictive re-acquisition work; the low time-locked figure mostly reflects frames in which no camera could see the beacon. The **decoy** scenario confirms the closed-loop example above across seeds. The **fast-target** scenario ($8^\circ/\text{s}$ random walk with hard turns) acquires in every run and keeps the lock once gained (retention $\approx 99\%$), but acquisition is slow and variable because the search has to catch a target that moves at two-thirds of the scan speed, and the constant-velocity filter lags the turns (≈ 9.6 px mean error). The **worst-case** scenario, with every disturbance at once, is where the current system fails: the beacon is acquired in 90% of runs but locked for only 10% of the time on average, because repeated occlusions, glints and strong turbulence keep sending it back to search. It is the main target for further work.

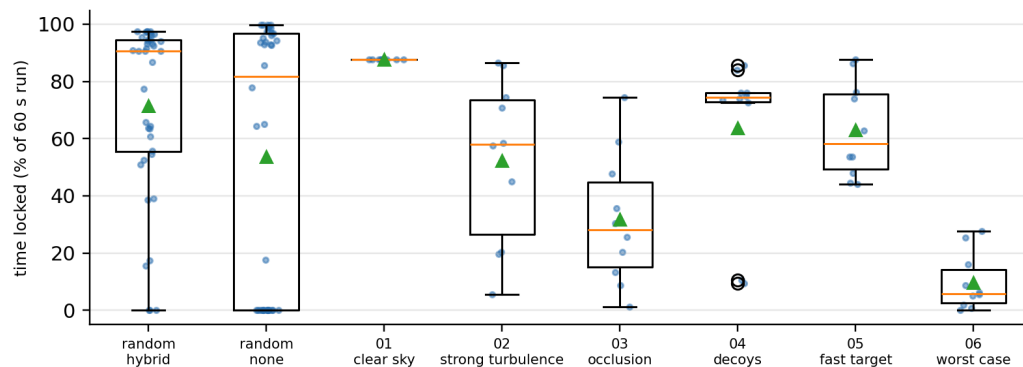


Figure 8. Distribution of the share of each 60 s run spent locked on the true beacon (box: quartiles, line: median, triangle: mean, dots: individual seeds).

11.4 Computational performance

The processing chain (detection, identification with batched CNN inference, tracking and control) takes on average 8 ms per 640×480 frame with the hybrid identifier on a laptop CPU, i.e. a throughput of roughly 120 frames/s, comfortably above the 30 frames/s requirement. Rendering the synthetic scene and applying the image-level disturbances is the more expensive half of the loop; it has been optimised with window-limited PSF evaluation, OpenCV routines for full-frame arithmetic, a pre-drawn noise bank and skipped cloud or glare work when nothing is in view. The desktop application runs the loop in real time with a deadline-based scheduler and can also run faster than real time.

12. User Interfaces and Tooling

12.1 Desktop application

`python app.py` starts a PySide6 application with a dark ‘mission control’ theme and a consistent colour language across all views (cyan = camera, red = beacon, amber = decoys, magenta = estimate, state colours SEARCH red, ACQUIRING orange, TRACKING green, COASTING yellow). It contains:

- the **camera feed** with detection boxes, per-candidate $P(\text{beacon})$ labels (green accepted, red rejected, yellow undecided), the selected candidate, the Kalman estimate with a 0.5 s velocity arrow, the dashed tracking window (ROI), the boresight and the handover circle;
- a **3D situational view** (pyqtgraph OpenGL): the pan-tilt terminal whose head turns with the gimbal, the sky dome, the simulated sky patch, the field-of-view pyramid and footprint coloured by tracker state, the history of where the camera has looked (the search spiral becomes visible), the pulsing beacon and its trail, decoys, clouds, stars and the estimate; with camera presets and optional auto-rotation;
- **live plots** of pointing error (log scale, locked periods shaded, handover line) and of the followed candidate's beacon score with the accept and reject thresholds;
- **controls** for scenario, seed, identification method, run length, simulation speed (0.5× to maximum), live sliders for turbulence, vibration and sensor noise, display options and report export; keyboard shortcuts Space, R, N and T; and **performance cards** with a description of the current random scene.

The simulation runs in the GUI thread paced by a 5 ms timer that advances as many frames as real time × speed requires, capped by a 28 ms budget per tick so the interface stays responsive. The views refresh at different rates (camera every tick, 3D view every 2nd, plots every 3rd, cards every 5th). A report is saved automatically at the end of a run.

12.2 Command line and tools

```
python main.py                                # OpenCV viewer, random scene
python main.py --config scenarios/04_decoys.yaml # fixed scenario
python main.py --headless --duration 120 --seed 7 # batch run, report only
python tools/generate_dataset.py                # labelled clips for the CNN
python tools/train_classifier.py                 # train, evaluate, export ONNX
python tools/compare_methods.py                 # none / blink / cnn / hybrid
python tools/evaluate.py --suite random --seeds 1-40 --methods hybrid none
```

The evaluation tool appends each run to `results.csv` immediately, so an interrupted evaluation resumes where it stopped, and it produces summary tables with Wilson confidence intervals for rates and normal-approximation intervals for means, breakdowns by scene property and box plots.

13. Limitations and Future Work

The following limitations were identified while reviewing the code and results:

- **Geometry.** The camera uses a linear small-angle projection and the world is a flat az/el patch. This is adequate for an 8° field of view and a $\pm 30^\circ$ sky, but a wide-FOV camera or near-zenith pointing would need a proper spherical (gnomonic) projection and gimbal kinematics.
- **Motion model.** The constant-velocity Kalman filter lags hard manoeuvres; the fast-target results show it. An interacting-multiple-model filter (constant velocity plus coordinated turn or constant acceleration) is the standard remedy.
- **Acquisition latency.** Identification needs at least 1.6 s of evidence before confirming. Shorter windows, a sequential probability ratio test with explicit error rates, or a higher-rate beacon code would shorten this.
- **Deployed model vs dataset.** The shipped model's report corresponds to the v1 dataset (no harmonic decoys, no glint samples). A v2 dataset with harmonic decoys has been generated; retraining and promoting a model from `models/candidate/` after checking it on the same validation protocol is the next step. Glints almost never survive long enough to form a 24-frame sample, so they are handled by the tracker's verification rather than the classifier.

- **Physical fidelity.** Turbulence is parameterised directly in pixels and log-intensity rather than derived from C_n^2 and path length; the sensor is 8-bit with a simple noise model. Linking these parameters to a link budget (range, wavelength, aperture) would make the simulator quantitatively predictive.
- **Scope of the loop.** Only the coarse stage is simulated; the fine-pointing handover is represented by a radius criterion. Closing the loop with a fast-steering-mirror model, and with the beacon on a real partner terminal that tracks back, would complete the PAT picture.
- **Evaluation size.** Scenario results are based on 10 seeds each and random scenes on 40; confidence intervals are therefore wide, and the between-seed spread (Figure 8) is as informative as the means.
- **Documentation drift.** The README states accept/reject thresholds of 0.7/0.3 and a 6 s blacklist, while the configuration uses 0.7/0.2 and 4 s; the configuration values are the ones used in this report.

14. Conclusion

The project delivers a complete, reproducible software test bench for the coarse stage of FSOC pointing, acquisition and tracking. The simulated camera and environment reproduce the effects that make coarse tracking hard, each switchable and scalable, and the closed loop implements the full PAT sequence from an open-loop pointing error through search, verification, tracking, coasting and re-acquisition. The key technical contribution is identity-aware tracking: candidates are followed over time, scored by a blink-frequency test and a compact spatio-temporal CNN, and the evidence is accumulated before the tracker accepts or rejects them. In random scenes this raises the acquisition rate from about 60% to over 90% and removes most false locks, at a processing cost well within real-time limits. Every run is logged against ground truth with strict metrics, and the evaluation tooling reports results over many seeds with confidence intervals, providing a sound basis for further tuning and for comparing new algorithms.

Appendix A. Key default parameters

Table 11. Default configuration (config/default.yaml).

Group	Parameter	Value
Simulation	Frame rate / default duration	30 fps / 60 s
Camera	Resolution, FOV, pixel scale	640×480 , $8^\circ \times 6^\circ$, $0.0125^\circ = 0.218$ mrad per px
Gimbal	Max rate / acceleration	$25^\circ/\text{s}$ / $80^\circ/\text{s}^2$
Beacon	Blink code	4 Hz, duty 0.7, off level 0.3
Detector	Global / ROI threshold	$\max(b + 8\sigma, 80) / \max(b + 5\sigma, 45)$, ROI ± 40 px
Identification	Clip, evidence memory, accept / reject	24 frames, 90 frames, 0.7 / 0.2
Identification	Verify limit, drop rule, blacklist	90 frames, < 0.35 for 30 frames, 4 s
Tracker	Acquire hits, hold, coast	5 hits, 6 frames, 60 frames or $\sigma > 1^\circ$
Tracker	q, R, gates	$3^\circ/\text{s}^2$, $(0.02^\circ)^2$, $\chi^2 < 11.6$ or $< 0.15^\circ$
Controller	Kp, Ki, Kd, feed-forward, anti-windup	5, 0.5, 0.05, 1.0, $\pm 2^\circ \cdot \text{s}$
Search	Spacing, speed, radius, passes, drift τ	4.8° , $12^\circ/\text{s}$, 15° , 4 then raster, 3 s
Metrics	Lock tolerance, handover radius	8 px, 10 px